

TS-RAG TIME: experiment guideline

1 Problem and hypotheses

Evaluate native TS-RAG on TIME while separating retrieval from context size and forecasting-backbone quality. Compare Bolt-512 with Bolt’s maximum context to measure the context restriction; compare TS-RAG with both Bolt controls to assess the retrieval wrapper, including maximum-context fallback. Chronos-T5 and Chronos-2 provide additional frozen controls. A validation mixture asks whether a pre-test win-frequency weight improves the stronger control. These are hypotheses, not experimentally supported conclusions.

2 Mathematical experiment

For a series of length T , official test interval length J and horizon H , the official origins are $t_j = T - J + jH$, for $j = 0, \dots, \lfloor J/H \rfloor - 1$. Rows are ordered by source item, variate, then origin, matching TIME’s univariate evaluator. The six frozen median methods are:

Method	Context / rule
Chronos-T5-base	$L = 512$, 20 forecast samples
Chronos-Bolt-base	$L = 512$
Chronos-Bolt-base	$L = 2048$
Chronos-2	$L = 8192$, no covariates or cross-learning
TS-RAG	$L = 512$, $K = 10$, Bolt-max fallback
Mixture	Frozen validation weight on Chronos-2 and TS-RAG

Available history is capped at L . Bolt maintains this cap throughout its official autoregressive quantile rollout. T5 retains its official sampled rollout. TS-RAG uses its separate 64-step median rollout.

2.1 Datastore and native forecasting

At real query t , eligible candidates are origins from all dataset items and variates. Each origin $s = 512 + j\Delta$ must satisfy $\tau_i(s + 64) \leq \tau_q(t)$, where τ maps item indices to calendar dates, with $\Delta = 1$ by default. No fitting-derived boundary exists. An optional maximum retains the most recent eligible windows in calendar order. Chronos-T5-base EOS embeddings have dimension 768. IndexFlatL2 requests 11 nearest candidates and retains the first 10, mirroring the released rule. Fetch 512 history plus 64 observed continuation points per neighbor. The ARM normalizes each full 576-point trajectory, encodes its 64-point future using a shared MLP, and combines these with the query Bolt decoder representation using attention and learned gates. The output projection emits 64 points per quantile.

Crop when $H \leq 64$. Otherwise append each median chunk, retain the last 512 points, re-embed, retrieve and repeat. Candidate eligibility stays anchored to the real t in every chunk. Earlier test observations may be retrieved by later real queries once the full native continuation is observed. Validation does not freeze the test datastore at the validation boundary. After selecting the mixture weight, keep that weight fixed and admit all observations available at each test query,

including validation-period dates, subject to stride, optional cap and the same calendar-causal boundary. Missing query history, fewer than 11 eligible neighbors, recorded extraction or inference errors, and non-finite native forecasts use the independently computed Bolt-max ($L = 2048$) forecast for the entire window/variant.

2.2 Retrieval ablation

Cross four binary axes. Scope compares all dataset items and variates with the query’s own item and variate. Date alignment and query-scale normalization are each off or on. Representation compares T5 EOS with IN lookback L2 at $L = 512$.

The grid contains 16 TS-RAG cells. The default uses all variates, no alignment, released neighbor normalization and T5. Each cell uses Bolt-max fallback. The four vanilla controls and default mixture give 21 method labels per task.

Aligned dates satisfy the same calendar phase as the query modulo the dataset’s observation-count alignment period. Copy Adapttime’s periods, with TIME seasonality as fallback and an explicit period override. Multiplied frequencies such as 15-minute sampling are accounted for; differently started items share a calendar cutoff. Alignment never relaxes the complete-neighbor causality condition.

Query-scale alignment uses neighbor lookback-only mean/std and query mean/std, with a 10^{-8} floor, to transform the full 576-point trajectory. For T5, embed aligned lookbacks per query and rollout chunk; these embeddings cannot reuse a query-independent T5 cache. Fuse aligned neighbors using the query’s Bolt normalization instead of their own full-trajectory normalization.

L2 selection caches instance-normalized lookbacks and uses squared finite-overlap distance scaled by $512/m$, requiring an overlap fraction of at least 0.8, matching Adapttime. It invokes no T5 selector. Alignment to query scale preserves IN L2 rankings but changes fusion, so retain these cells in the full factorial grid.

2.3 Validation and mixing

The optional validation interval of length V ends at $T - J$; origins use the same horizon stride as testing. Default V is the dataset’s configured TIME validation length. $V = 0$ disables weight estimation. If $0 < V < H$, or the interval leaves no observed prefix, validation is unavailable and the test interval remains unchanged. There is no training or fitting split.

For each validation variate/date, let c be the last 512 available history points and define the query scale

$$q = \max\{\text{std}(c), 10^{-8}\}.$$

Divide the finite-step mean squared error by q^2 . Require finite paired scores from Chronos-2 and the TS-RAG method, including its explicit fallback. Average over supported variates per series/date. With n usable dates and w TS-RAG wins (ties count one half), freeze $p = (1 + w)/(2 + n)$ and predict $(1 - p)C + pR$, where C and R are Chronos-2 and TS-RAG medians. If $n = 0$, use $p = 0$ explicitly. An invalid combined test forecast falls back to Bolt-max on that complete cell.

2.4 Datasets, randomness and evaluation

Consume prepared TIME saved-Arrow data using the official dataset settings. The full run includes all available short/medium/long terms except `Coastal.T.S/5T`, `current.velocity/20T`, `azure2019.D/5T` and `azure2019.I/5T`, giving 90 tasks. Explicit selection may change this scope. The remote test mode selects only `SG.Weather/D`, short. A single seed (default 0) controls Python, NumPy and Torch randomness, including T5 forecast sampling; no independent component seeds exist. Prepared tensor consumers apply no CSV exclusion or missing-value policy again.

The selected completed Seasonal Naive artifact defines finite target steps and metric cells requiring finite Seasonal MASE. Foundation controls and fallback predictions must be finite on

required cells. All medians enter the standard TIME saver as a single 0.5 quantile. Report MSE, MAE, RMSE, MAPE, sMAPE, MASE, ND and deterministic single-quantile CRPS, retaining finite/grid/ total cell counts. Reject additional coverage loss relative to Seasonal.

For finite task metric cells $a_1, \dots, a_m$, report

$$\bar{a} = \frac{1}{m} \sum_i a_i, \quad \text{var}(a) = \frac{1}{m} \sum_i (a_i - \bar{a})^2, \quad \text{std}(a) = \sqrt{\text{var}(a)}.$$

This is population dispersion, with ddof 0. Scaled MASE divides the model task mean/std by matched Seasonal mean, and its variance by the square of that mean. The separate MASE variance ratio divides model variance by Seasonal variance; an unavailable/zero denominator remains undefined. Overall reports include arithmetic task means and equal-dataset scaled means.

3 Implementing code and execution

`data/windows.py` owns references and causal candidates; `model_loading/` owns official controls and strict ARM loading; `external_models/tsrag/` owns source-adapted native computation; `proposal/mixture.py` owns the validation rule; `pipeline/workflow.py` composes stages; `evaluation/` and `results/` own metrics and reports. These paths are under `src/timebench/`.

1. Prepare datastore, validation and official test references.
2. Compute and persist the four independent vanilla controls.
3. Extract frozen T5 EOS and instance-normalized lookback caches.
4. Predict native TS-RAG with explicit Bolt-max fallback.
5. Estimate/freeze the validation weight and produce mixture medians.
6. Evaluate all six main methods or 21 ablation methods on the selected Seasonal grid.
7. Write matched comparison and source-manifest reports.

Run `bash scripts/submit_experiment.sh dgx` from the project root after the selected Seasonal store is complete. Hydra configuration is

```
src/timebench/conf/experiment.yaml
```

Use `validation_length=0` for no validation, and explicit `datasets=[...]` / `terms=[...]` overrides for a subset. Experiment and Seasonal launchers live under `scripts/`; they submit the root Slurm fronts and delegate implementations to `src/slurm/`. For the complete retrieval grid run:

```
bash scripts/submit_ablation.sh dgx
```

Main and ablation runs reuse matching data, caches and default predictions. One allocation runs the visible stages sequentially.

All Selenia fronts request one GPU, partition `an`, QoS `an_preemptable`, exclusive allocation and WCKey `P12CU:DATASCIENCE`. Their defaults are one node, one task, eight CPUs, 80 GB memory and 23 hours; cluster requeue remains enabled.

Every task phase has a schema-1 plain scientific manifest allocated before expensive work. Exact completed phases are reusable; interrupted tasks restart from their beginning. Producer resolution embeds complete scientific configuration and the seed, while storage paths and source revisions are provenance. Completion follows successful owning `srun` and declared nonempty artifacts. No source, dataset or checkpoint file hashes exist. Canonical float32 predictions, fallback masks/reasons and weights live below `outputs/tsrag/`; logs live below `logs/`. Scheduler

runtimes enforce the owning project’s artifact roots, ignoring copied or inherited artifact aliases while retaining shared dataset and weight settings. Reports include retrieval axes, alignment periods and fallback methods, and record all input evaluation, prediction and Seasonal manifests. Consumers must select equivalent completed computations and read canonical artifacts in place. Reported fallback counts, rates and aggregate causes use the test split only, which is the evaluated split. Validation masks and per-row reasons remain split-specific diagnostics for mixture estimation and are excluded from those reported totals.

Timings separate datastore extraction and query work. TS-RAG reuses cached Bolt-max fallback forecasts; its measured inference time excludes computing that separate control. Mixture totals include both component times and mixing. These are pipeline measurements, not independent fresh-process latency. The new growing datastore and fallback invalidate direct reuse of Adapttime’s historical TS-RAG forecasts: run the full current computation afresh.